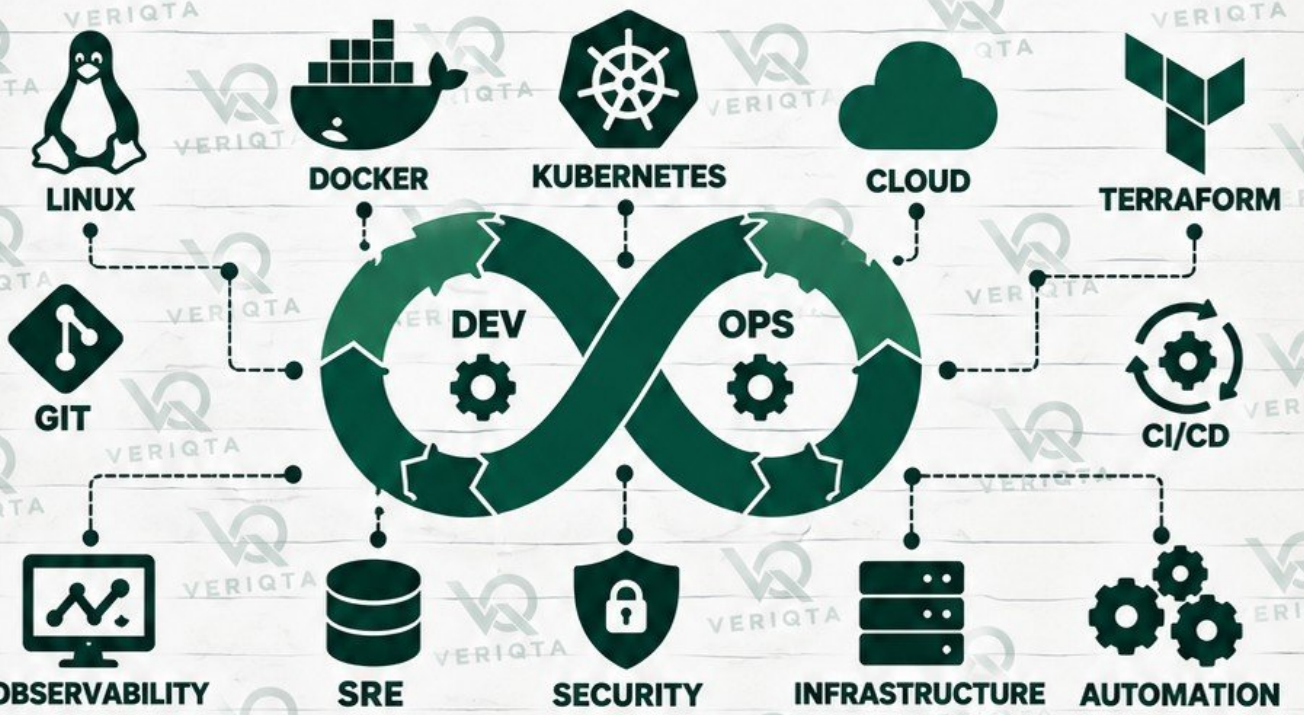
VERIQTA

50 Most Repeated DevOps Interview Questions

(2026 Edition)



@veriqta | @veriqta | @veriqta | @veriqta | @veriqta | @veriqta

Linux Performance and System Troubleshooting

Q1 A production Linux server has a high load average, but CPU utilization is low. What would you investigate first?

What It Means

- **Load average** shows the number of processes waiting for CPU, disk I/O or other resources.
- If CPU usage is low, the processes are likely waiting on I/O, memory, locks or other resources.
- High load does not always mean high CPU usage.

Key Areas to Investigate

1. Check disk I/O wait.
2. Check for processes in uninterruptible sleep (D state).
3. Check memory pressure or swapping.
4. Check for filesystem or network I/O bottlenecks.
5. Identify long-running processes and their wait reason.

Useful Commands

```
# Check load and CPU
uptime
top

# Check I/O wait
vmstat 1 5
iostat -xz 1 5

# Find processes in D state
ps -eo pid,state,cmd | grep ' D '

# Check disk activity
iotop
lsblk
df -h
```

Q2 How would you diagnose a Linux server experiencing memory pressure without immediately restarting services?

Symptoms

- High memory usage
- Increased swap usage
- Slow application response
- Kernel OOM killer activity
- Processes being killed

Investigation Steps

1. Check memory and swap usage.
2. Identify which processes use memory.
3. Look for signs of memory leak or abnormal growth.
4. Check kernel logs for OOM events.
5. Investigate filesystem cache and page cache usage.

Useful Commands

```
# Memory usage
free -h
vmstat 1 5
swapon --show

# Find top memory-consuming processes
top
htop
ps aux --sort=-%mem | head

# Check OOM killer logs
dmesg | grep -i oom
journalctl -k | grep -i oom
```

Production Tip: Identify and stop non-critical processes, reduce memory usage, or increase available memory. Restart services only after confirming the root cause.

Q3 A production application reports "Too many open files." How would you identify the cause and safely resolve it?

What It Means

- The process has reached the maximum number of file descriptors (FDs) it can open.
- Can be caused by file descriptor leaks, high connection count, or low system limits.

Investigation Steps

1. Check the error in application logs.
2. Find the process and its open file count.
3. Check system and user limits.
4. Identify the type of open files (sockets, files, etc.).
5. Determine if the application is leaking file descriptors.

Useful Commands

```
# Find process open file count
lsof -p <PID> | wc -l

# See open files by type
lsof -p <PID> | haad

# Check system limits
ulimit -n
cat /proc/<PID>/limits

# Check system-wide usage
cat /proc/sys/fs/file-nr
```

Resolution Options

- Increase file descriptor limits (ulimit, /etc/security/limits.conf, systemd).
- Fix application code if there is a file descriptor leak.
- Use connection pooling and ensure files and sockets are properly closed.
- Monitor open file count over time.

Example: Increase Limits

```
# Temporary (current session)
ulimit -n 65536

# Permanent (systemd service)
LimitNOFILE=65536
```

Linux Processes, Services and Resource Isolation

Q4 How do you troubleshoot a systemd service that repeatedly crashes even though it starts successfully when run manually?

Common Causes

1. Different environment variables in systemd.
2. Different working directory (WorkingDirectory).
3. Incorrect user or permissions.
4. Missing dependencies or network not ready.
5. Resource limits (memory, CPU, file descriptors).
6. Timeout or restart policy killing the service.

Investigation Steps

1. Check service status and recent logs.
2. Compare systemd environment with manual shell environment.
3. Check the effective user, working directory and permissions.
4. Review service configuration.
5. Check resource limits and cgroup settings.
6. Identify crash reason from logs, core dumps or exit codes.

Useful Commands

```
# Check service status and logs
sudo systemctl status <service>
sudo journalctl -u <service> -n 100

# View service configuration
systemctl cat <service>

# Check environment variables
systemctl show <service> | grep -i env

# Check resource limits
systemctl show <service> | grep -i limit

# Run service as systemd user manually
sudo -u <user> /path/to/binary

# Check recent restarts
systemctl status <service> --no-pager
```

Key Configuration Items

- `User=` and `Group=`
- `WorkingDirectory=`
- `Environment=`
- `ExecStart=`
- `Restart=` and `RestartSec=`
- `LimitNOFILE=`, `LimitNPROC=`
- `MemoryMax=`, `CPUQuota=`
- `Requires=` and `After=`

Example Service Snippet

```
[Service]
User=appuser
WorkingDirectory=/app
Environment=ENV=prod
ExecStart=/app/bin/server
Restart=on-failure
RestartSec=5
LimitNOFILE=65536
MemoryMax=1G
CPUQuota=50%
```

Production Tip

- Always check journal logs for the real failure reason.
- Compare systemd environment with your shell environment.
- Verify user permissions, working directory and required dependencies.
- Check resource limits and cgroup constraints.
- Use restart policies carefully to avoid crash loops.

Q5 How do Linux cgroups and namespaces affect container resource isolation, and what limitations should engineers understand?

What They Do

- Namespaces isolate resources so processes in a container see a separate view of the system.
- Cgroups (control groups) limit, account and control resource usage.
- Together they provide process, network, filesystem and resource isolation for containers.

Key Namespaces

PID	Isolates process IDs
NET	Isolates network stack
MNT	Isolates filesystem mounts
UTS	Isolates hostname and domain
IPC	Isolates inter-process communication
USER	Isolates user and group IDs

Key Cgroups (v2)

CPU	Limits and shares CPU resources
Memory	Limits memory usage and controls OOM
IO	Limits disk I/O
PIDs	Limits number of processes
Blkio	Controls block I/O (legacy name)
Devices	Restricts device access

Important Limitations

1. Not a security boundary. Root in a container can sometimes escape if the host is misconfigured.
2. Kernel resources are shared (kernel version, kernel bugs).
3. Some resources (e.g. certain hardware, network performance) are not fully isolated.
4. Misconfigured cgroups can still allow noisy-neighbor issues.
5. Privileged containers or excessive capabilities reduce isolation.

Production Considerations

- Use cgroups to set CPU and memory limits for containers.
- Monitor container resource usage (CPU, memory, I/O, PIDs).
- Use namespaces to isolate processes, network and filesystem.
- Avoid running containers in privileged mode.
- Keep the host kernel updated to reduce escape risks.
- Combine with security controls (Seccomp, AppArmor/SELinux) for stronger isolation.

DNS, TCP and Production Network Failures

Q6 A service is reachable by IP address but intermittently fails through its DNS name. How would you investigate?

Possible Causes

1. DNS returns different IPs (some unhealthy).
2. DNS records have low/short TTL causing frequent changes.
3. Partial DNS resolution failure or timeouts.
4. Client or local DNS cache issues.
5. Load balancer or some backend instances unhealthy.

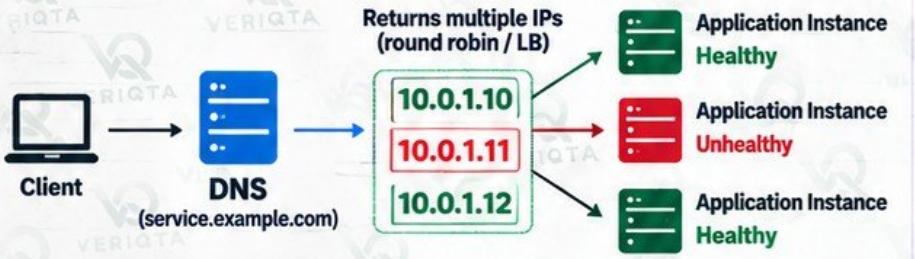
Investigation Steps

1. Compare DNS resolution results over time and from different locations.
2. Check if returned IPs are healthy.
3. Verify DNS TTL and record configuration.
4. Test direct connectivity to each resolved IP.
5. Check local DNS cache and resolver logs.
6. Identify if the issue is on client, network, DNS provider or load balancer.

Useful Commands

```
# Check DNS resolution
nslookup service.example.com
dig service.example.com
dig +short service.example.com
dig service.example.com +ttlunits
# Test each resolved IP
curl -I http://<resolved-ip>
ping <resolved-ip>
# Check local DNS cache (Linux)
systemd-resolve --statistics
cat /etc/resolv.conf
```

Typical Architecture



Production Tip

- Check DNS from multiple locations (e.g. client, server, and another region) to identify inconsistent responses.
- Monitor DNS query latency, resolution failures and TTL values.
- Ensure health checks remove unhealthy instances from DNS or load balancer.

Q7 How would you troubleshoot intermittent TCP connection timeouts between two services?

Common Causes

1. Network congestion or packet loss.
2. Firewall or security group rules.
3. Load balancer or NAT timeout.
4. Asymmetric routing.
5. MTU mismatch causing dropped packets.
6. Server resource exhaustion (CPU, memory).
7. Application not accepting connections.

Investigation Steps

1. Confirm the timeout pattern and frequency.
2. Check network connectivity and latency.
3. Look for packet loss or retransmissions.
4. Verify firewall, security groups and ACLs.
5. Test from both sides and compare results.
6. Check server and application logs.
7. Identify recent changes (deployments, network or security updates).

Useful Commands

```
# Test connectivity and latency
ping <host>
# Check TCP connection
nc -vz <host> <port>
telnet <host> <port>
# Trace route
traceroute <host>
# Inspect network issues
mtr <host>
ss -s
tcpdump -nn host <host> and port <port>
```

Q8 What causes connection pool exhaustion, and how can retries make the problem worse?

Common Causes

1. Too many concurrent requests.
2. Slow responses from downstream service.
3. Connections not released back to the pool.
4. Long-lived or idle connections.
5. Pool size configured too small.
6. Sudden traffic spike.

How Retries Make It Worse

1. When the service is slow, requests time out.
2. Clients retry immediately or with short delay.
3. Retries create more concurrent connections.
4. Connection pool fills up faster.
5. More timeouts occur, causing even more retries (retry storm).
6. This can cascade and take down the service.

Best Practices

- Set appropriate connection pool size.
- Use timeouts and retry with exponential backoff and jitter.
- Implement circuit breakers.
- Ensure connections are properly closed.
- Monitor pool usage, pending requests and connection errors.
- Scale the service or optimize slow queries.

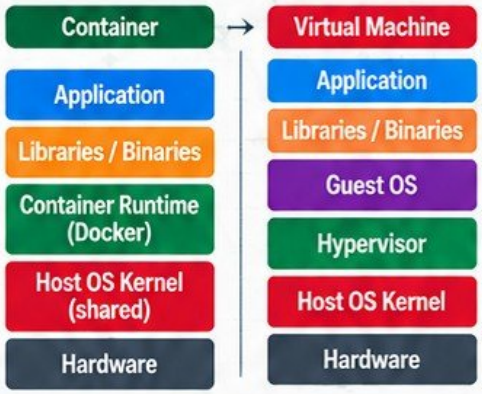
Docker Internals and Container Troubleshooting

Q9 What happens internally when a container starts, and how does it differ from starting a virtual machine?

Container Start Flow

1. Docker Engine pulls the image (if not present).
2. Image layers are mounted using a union filesystem (OverlayFS).
3. A new container process is created using Linux namespaces (PID, NET, MNT, UTS, IPC).
4. cgroups apply resource limits (CPU, memory, I/O).
5. Container starts with the specified command (ENTRYPOINT/CMD).
6. Shares the host kernel but runs in isolated namespaces.

Container vs Virtual Machine



Key Differences

1. Containers share the host kernel; VMs have a full guest OS.
2. Containers start in seconds; VMs take longer to boot.
3. Containers are lightweight; VMs are heavier.
4. Containers use namespaces and cgroups; VMs use hardware virtualization.
5. Containers have smaller resource overhead.

Q10 A container runs successfully locally but repeatedly exits in production. How would you diagnose the failure?

Common Causes

1. Environment variable differences.
2. Missing configuration or secrets.
3. Different file paths or mounted volumes.
4. Permission issues (user, file, directory).
5. Resource limits (CPU, memory, disk).
6. External dependency not reachable.
7. Different Docker image version or build.
8. Health check or startup script failure.

Investigation Steps

1. Check container logs.
2. Inspect container exit code.
3. Compare environment variables and configuration.
4. Verify mounted volumes and permissions.
5. Check resource limits and node conditions.
6. Test external dependencies (DB, API, DNS).
7. Compare image version and build process.
8. Reproduce the issue with the same image and environment settings.

Useful Commands

```
# Check container logs
docker logs <container_id>

# Check exit code
docker inspect -f '{{.State.ExitCode}}'
<container_id>

# Inspect environment variables
docker inspect <container_id> | grep -i
Eny -A 10

# Check resource limits
docker inspect <container_id> | grep -i
'HostConfig' -A 10

# Run an interactive shell for debugging
docker run --rm -it <image> sh
```

Q11 How would you reduce Docker image size while maintaining security, reproducibility, and build performance?

Best Practices

1. Use a minimal base image (e.g. distroless or Alpine).
2. Use multi-stage builds.
3. Install only required packages.
4. Remove build-time dependencies and caches.
5. Use `.dockerignore` to exclude unnecessary files.
6. Combine RUN commands to reduce layers.
7. Use specific, versioned base images for reproducibility.
8. Scan images for vulnerabilities.
9. Use non-root user and least-privilege configuration.
10. Optimize build cache and layer ordering for performance.

Example: Multi-Stage Dockerfile

```
# Build stage
FROM golang:1.22 AS builder
WORKDIR /app
COPY . .
RUN go build -o app

# Runtime stage
FROM gcr.io/distroless/base
COPY --from=builder /app/app /app
USER nonroot
CMD ["/app"]
```

Additional Tips

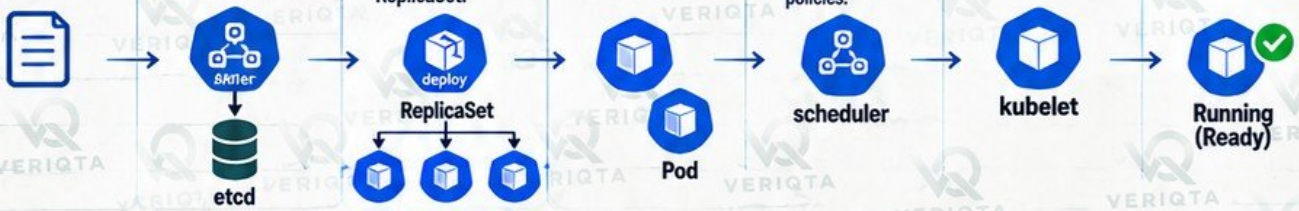
- Use distroless images for smaller attack surface.
- Remove package manager caches (apt, yum, apk).
- Clean unnecessary files and logs.
- Use reproducible builds with pinned versions.
- Scan with tools like Trivy or Docker Scout.
- Keep images small, signed and stored in a trusted registry.

Kubernetes Control Plane and Scheduling

Q12 What happens inside Kubernetes from the moment a Deployment is submitted until its Pods become ready?

End-to-End Flow

1. You submit a Deployment (kubectl apply).
2. API Server validates and stores the object in etcd.
3. Deployment controller notices the new Deployment and creates a ReplicaSet.
4. ReplicaSet creates the required Pods.
5. Scheduler assigns each Pod to a node based on resources, constraints and policies.
6. Kubelet on the node pulls the image and starts the containers.
7. Readiness probe succeeds and the Pod becomes Ready.



Q13 A Pod remains Pending even though the cluster has available CPU and memory. What would you investigate?

Common Causes

1. Node selector, affinity or anti-affinity rules not matching any node.
2. Taints and tolerations preventing scheduling.
3. Insufficient resources due to requests, not actual usage.
4. PersistentVolumeClaim (PVC) not bound.
5. Image pull issues.
6. Container runtime or node problems.
7. Pod security policies or admission controllers.
8. Topology, zone or network constraints.

Investigation Steps

1. Check pod events and description.
2. Verify node selectors, affinity, tolerations and topology constraints.
3. Confirm PVC status and storage class.
4. Check image pull errors.
5. Review resource requests vs node allocatable resources.
6. Check admission controller and security policies.
7. Look at cluster and node health.
8. Review relevant logs (scheduler, kubelet, controller-manager).

Useful Commands

```
# Describe the pod
kubectl describe pod <pod-name>

# Check events
kubectl get events --sort-by=.lastTimestamp

# Check nodes and resources
kubectl get nodes
kubectl describe node <node-name>

# Check PVC
kubectl get pvc
kubectl describe pvc <pvc-name>

# Check scheduler logs (control plane)
kubectl -n kube-system logs \
  -l component=kube-scheduler

# Check image pull issues
kubectl describe pod <pod-name> | grep -i image

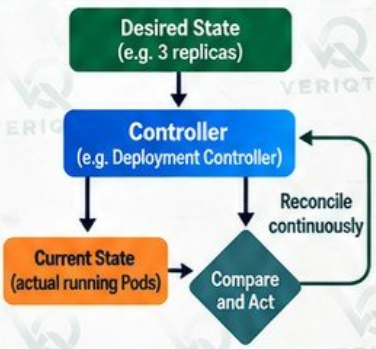
# Check taints and tolerations
kubectl describe node <node-name> | grep Taints
```

Q14 How does Kubernetes reconciliation work, and what happens when a controller or API server becomes unavailable?

How Reconciliation Works

1. You define the desired state (e.g. 3 replicas).
2. The controller watches for changes in the API server (desired vs current state).
3. It continuously compares the current state with the desired state.
4. If there is a aifference, it takes action to move the current state toward the desired state.
5. This loop runs continuously until the desired state is achieved.

Reconciliation Flow



If a Component Becomes Unavailable

Controller Unavailable

- Reconciliation stops for that resource.
- Existing Pods continue running.
- No new Pods are created or replaced until the controller is back.
- Once it recovers, it detects the difference and reconciles the state.

API Server Unavailable

- Controllers cannot read or write desired state.
- No new changes are processed.
- Existing workloads keep running.
- Once the API server is available, controllers resume reconciliation.

Important: Kubernetes controllers are designed to be resilient. When a component recovers, reconciliation continues automatically to bring the current state back to the desired state.

Kubernetes Networking and Service Discovery

Q15 A Kubernetes Service exists, but applications cannot reach its backend Pods. How would you troubleshoot?

Common Causes

1. Pods are not ready or failing health checks.
2. Incorrect selector (labels do not match Pods).
3. Endpoints/EndpointSlice is empty.
4. Network policies blocking traffic.
5. DNS resolution issues.
6. CNI networking problem.
7. Service type or port configuration error.
8. Firewall or security group rules (for external access).
9. Application binding to localhost only.
10. Pod is running but not listening on the expected port.

Troubleshooting Steps

1. Check Service, endpoints and Pod status.
2. Verify labels and selectors match.
3. Check Pod logs and readiness probes.
4. Test connectivity from inside the cluster.
5. Review NetworkPolicies and firewall rules.
6. Verify DNS resolution.
7. Confirm correct service type and ports.
8. Inspect CNI and node networking.
9. Check application listening address and port.
10. Compare with a working environment.

Useful Commands

```
# Check service details
kubectl get svc <service-name> -o wide
kubectl describe svc <service-name>

# Check endpoints
kubectl get endpoints <service-name>
kubectl get endpointslices -l
kubernetes.io/service-name=<service-name>

# Check pods and logs
kubectl get pods -l <label> -o wide
kubectl logs <pod-name>

# Test DNS and connectivity from a pod
kubectl exec -it <pod> -- nslookup
<service-name>
kubectl exec -it <pod> -- curl
http://<service-name>:<port>

# Check network policies
kubectl get networkpolicy
kubectl describe networkpolicy <name>
```

Q16 How do Kubernetes DNS, CNI networking, and Service routing interact during a request?

Request Flow



Key Components and Roles

1. **CoreDNS:** Resolves service names to ClusterIP.
2. **ClusterIP:** Virtual IP for the service.
3. **kube-proxy:** Routes traffic to backend Pods (iptables or IPVS).
4. **CNI (Container Network Interface):** Provides Pod networking and routing between nodes.
5. **Endpoints/EndpointSlice:** Dynamically updated list of healthy Pods.
6. **Service routing:** Load balances traffic to healthy backend Pods.

Q17 How would you diagnose intermittent traffic failures during a Kubernetes rolling deployment?

Possible Causes

1. Not enough ready replicas during rollout.
2. Readiness probe misconfigured or failing.
3. Pods becoming unready due to startup time.
4. Short connection timeouts or aggressive retries.
5. Session affinity or sticky sessions.
6. Load balancer removing/adding endpoints slowly.
7. Network policies or CNI issues.
8. DNS caching or endpoint propagation delay.
9. Application bugs triggered by new version.
10. Resource pressure (CPU, memory) causing instability.

Investigation Steps

1. Check rollout status and pod readiness.
2. Monitor deployments, events and logs.
3. Verify readiness and liveness probes.
4. Check the number of ready pods over time.
5. Inspect service endpoints during rollout.
6. Review application and ingress/load balancer logs.
7. Check for increased latency, errors or timeouts.
8. Compare behavior between old and new versions.
9. Verify resource usage and node conditions.
10. Reproduce the issue and identify patterns.

Useful Commands

```
# Check rollout status
kubectl rollout status deployment/<name>
kubectl get pods -l app=<name> -o wide

# Check events
kubectl get events --sort-by=.lastTimestamp

# Check readiness probe and pod status
kubectl describe pod <pod-name>

# Watch endpoints during rollout
kubectl get endpoints <service-name> -w

# Check logs
kubectl logs <pod-name>

# Check resource usage
kubectl top pods
# Check ingress or load balancer logs
kubectl logs -n <ns> <ingress-pod>
```

Production Tip: During rolling deployments, ensure sufficient ready replicas, correct readiness probes, and monitor error rates, latency and endpoint changes in real time.

Kubernetes Resource Management and Scaling

Q18 A Pod is repeatedly OOMKilled despite apparently low average memory usage. How would you investigate?

Common Causes

1. Memory spikes (short-lived) not shown in average metrics.
2. Container memory limit is too low.
3. Memory leak in the application.
4. Large in-memory cache or data structure.
5. JVM or language runtime overhead.
6. Multiple containers in the same Pod competing for memory.
7. Kernel memory usage (e.g. page cache). not visible in container metrics.

Investigation Steps

1. Check Pod events for OOMKilled.
2. Review container memory limits and usage.
3. Inspect memory usage over time (look for spikes).
4. Check application logs for memory errors.
5. Use kubectl top pod and container metrics.
6. Compare memory usage vs limit.
7. Inspect application metrics (heap, non-heap).
8. Check node memory pressure and system logs.

Useful Commands

```
# Check pod events
kubectl describe pod <pod-name>

# Check container resource usage
kubectl top pod <pod-name>

# View container logs
kubectl logs <pod-name> -c <container>

# Describe node for memory pressure
kubectl describe node <node-name>

# Check dmesg for OOM events
dmesg | grep -i kill

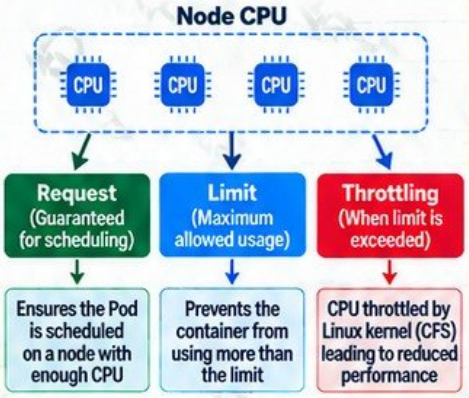
# Monitor memory usage over time
kubectl top pod <pod-name> --containers
```

Q19 How do CPU requests, CPU limits, and throttling affect Kubernetes application performance?

Key Concepts

1. CPU Request: Minimum amount of CPU guaranteed for scheduling.
2. CPU Limit: Maximum amount of CPU a container can use.
3. Throttling: When a container tries to use more than its CPU limit, the Linux kernel (CFS) throttles it.
4. Requests affect scheduling, not runtime throttling.
5. Limits prevent noisy neighbors but can cause performance issues if set too low.

How It Works



Impact on Performance

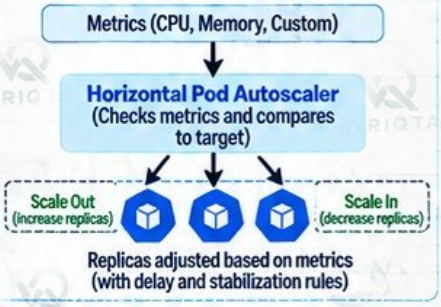
1. If request is too low, Pod may be scheduled on a busy node, causing CPU contention.
2. If limit is too low, the container can be throttled, causing higher response times and lower throughput.
3. If request equals limit, the container gets Guaranteed QoS (best performance).
4. CPU throttling can cause latency spikes, timeouts and reduced performance.
5. Proper sizing helps with stable performance and fair resource sharing.

Q20 Why might the Horizontal Pod Autoscaler fail to improve performance during a sudden traffic spike?

Common Causes

1. HPA metric delay (metrics are not real-time).
2. High stabilization window (delays scaling).
3. Incorrect metric configuration (e.g. wrong metric or target value).
4. Pods take too long to start (slow initialization).
5. Max replicas limit reached.
6. Metrics server or custom metrics adapter issues.
7. Application not stateless or not horizontally scalable (e.g. single DB, external dependency).

How HPA Works



Troubleshooting Steps

1. Check current HPA configuration.
2. Verify metrics (CPU, memory or custom) are correct and up to date.
3. Check metrics server or custom metrics adapter health.
4. Review HPA events and status.
5. Check pod startup time and readiness probes.
6. Verify maxReplicas is not too low.
7. Look for application bottlenecks (e.g. database, external services).

Useful Commands

```
# Check HPA status
kubectl get hpa
kubectl describe hpa <hpa-name>

# Check metrics
kubectl top pods

# Check events
kubectl get events --sort-by=.lastTimestamp
```

Production Tips

- Use appropriate and responsive metrics (e.g. custom metrics for request rate).
- Set reasonable target values.
- Reduce stabilization window if needed.
- Ensure pods start quickly (optimize image and probes).
- Verify metrics server and monitoring.
- Set an appropriate maxReplicas.
- Test scaling behavior under load.
- Ensure the application is stateless and can scale horizontally.

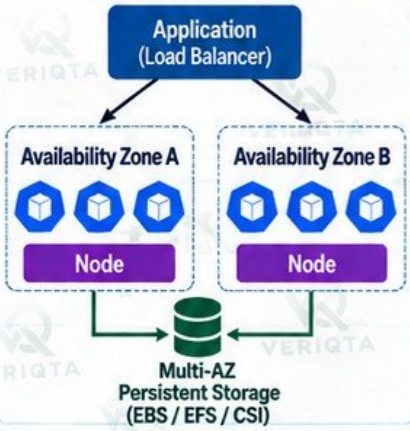
Kubernetes Availability and Incident Recovery

Q21 How would you design Kubernetes workloads to tolerate node failures and Availability Zone disruptions?

Key Design Strategies

1. Use multiple replicas across nodes and Availability Zones.
2. Use anti-affinity or topology spread constraints.
3. Distribute workloads across zones/regions.
4. Use managed node groups with auto-scaling and self-healing.
5. Use persistent storage with multi-AZ (e.g. EBS, EFS, CSI with replication).
6. Use stateless design where possible.
7. Configure PodDisruptionBudgets (PDB).
8. Test failure scenarios (node drain, AZ outage).

Example Architecture



Useful Commands

```
# Check node distribution
kubectl get pods -o wide

# Check topology spread constraints
kubectl get deploy <name> -o yaml

# Check PodDisruptionBudget
kubectl get pdb

# View node zones
kubectl get nodes -L topology.kubernetes.io/zone

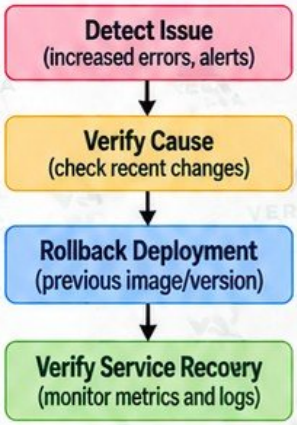
# Simulate node drain
kubectl drain <node-name> --ignore-daemonsets
--delete-emptydir-data --force
```

Q22 A Kubernetes deployment causes widespread application failures. What is your safest recovery strategy?

Recovery Strategy Steps

1. Assess the impact and scope (which services, which environment).
2. Identify the cause (recent deployment, config change, external dependency).
3. Roll back to the last known good version (or previous image).
4. Use progressive rollback (e.g. canary/partial) if needed.
5. Verify dependent services and database compatibility.
6. Monitor metrics, logs and error rates after rollback.
7. Communicate status to stakeholders.
8. Perform a post-incident review and prevent recurrence.

Rollback Approaches



Useful Commands

```
# View rollout history
kubectl rollout history deployment/<name>

# Roll back to previous version
kubectl rollout undo deployment/<name>

# Roll back to a specific revision
kubectl rollout undo deployment/<name>
--to-revision=<number>

# Check deployment status
kubectl rollout status deployment/<name>

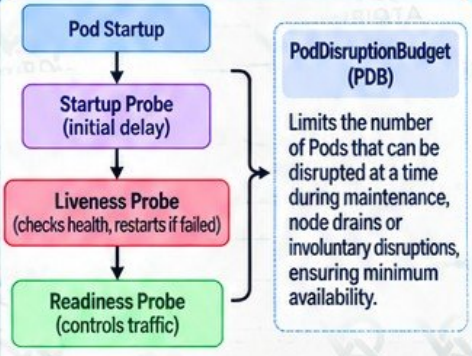
# View events and logs
kubectl get events --sort-by=.lastTimestamp
kubectl logs <pod-name>
```

Q23 How do readiness probes, liveness probes, and PodDisruptionBudgets affect production availability?

Probe Types

Probe Type	Purpose	Effect on Availability
Readiness Probe	Checks if the application is ready to receive traffic.	Prevents traffic from being sent to Pods that are not ready, reducing errors during startup or degraded state.
Liveness Probe	Checks if the application is healthy and running.	Restarts the container if it becomes unhealthy, improving self-healing but can cause outages if misconfigured.
Startup Probe (optional)	Gives the application time to start before liveness checks begin.	Prevents premature restarts for slow-starting applications.

How They Work Together



Key Benefits

1. Readiness probes prevent traffic to unready Pods.
2. Liveness probes enable self-healing by restarting unhealthy containers.
3. Startup probes avoid unnecessary restarts for slow applications.
4. PodDisruptionBudgets maintain minimum availability during updates or node maintenance.
5. Proper configuration reduces downtime and error rates.
6. Together, they improve application resilience and stability in production.

VERIQTA

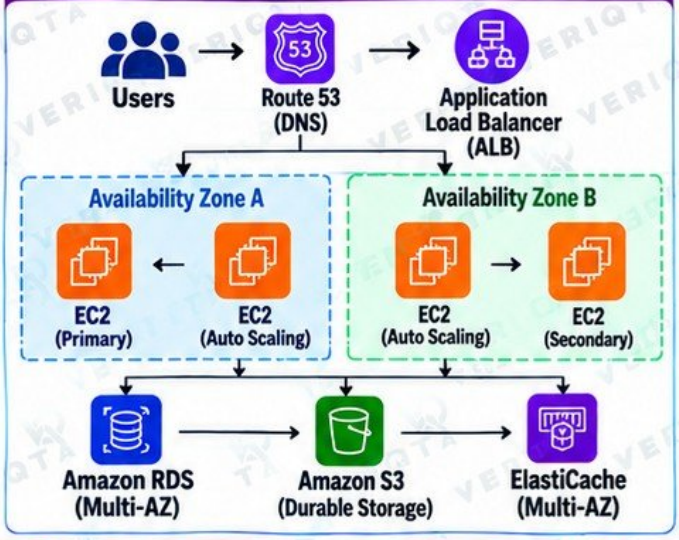
AWS Architecture and High Availability

Q24. How would you design a highly available production application across multiple AWS Availability Zones?

KEY DESIGN PRINCIPLES

- Deploy across multiple Availability Zones (AZs).
- Use a Load Balancer to distribute traffic across healthy resources.
- Keep the application stateless or use shared storage.
- Use managed services with multi-AZ support.
- Implement health checks, auto scaling and failover.
- Design for failure and test it regularly.

HIGHLY AVAILABLE ARCHITECTURE (MULTI-AZ)



KEY AWS SERVICES

- **Route 53** — DNS and failover
- **Application Load Balancer** — Distribute traffic
- **EC2 Auto Scaling** — Scale across multiple AZs
- **RDS Multi-AZ** — High availability database
- **S3** — Durable and highly available storage
- **ElastiCache Multi-AZ** — High availability caching

Q25. What are the architectural and operational trade-offs between a NAT Gateway, VPC endpoints, and direct internet access?

NAT Gateway

Private subnet resources access the internet (outbound only).

ADVANTAGES
- Simple to set up and manage
- Works for any internet destination
- No inbound access

DISADVANTAGES
- Higher cost (per hour + data)
- Managed per AZ (needs one per AZ)
- Not suitable for high volume traffic

BEST USE CASE

Private resources need general internet access (e.g. package updates).

VPC Endpoints

Private access to AWS services without using the internet.

ADVANTAGES
- More secure (no internet exposure)
- Lower cost than NAT for AWS services
- Uses AWS private network
- Supports S3 (Gateway) and many services (Interface)

DISADVANTAGES
- Works only for supported AWS services
- Does not provide general internet access
- Some endpoints (Interface) cost per hour

BEST USE CASE

Access to AWS services like S3, DynamoDB, ECR, CloudWatch, etc. from private subnets.

Direct Internet Access

Resources in public subnets access the internet directly.

ADVANTAGES
- Simple and low cost
- Useful for public resources
- No additional components required

DISADVANTAGES
- Exposes resources to the internet
- Not suitable for sensitive workloads
- Requires security controls (security groups, NACLs, WAF)

BEST USE CASE

Public resources like web servers, bastion hosts or when direct access is required.

Q26. How would you investigate an application outage caused by an AWS dependency or regional service disruption?

1. IDENTIFY THE SCOPE
- Check what is failing
- Impact to users
- Which services are affected

→

2. CHECK AWS SERVICE HEALTH
- Review AWS Health Dashboard
- Check for regional service issues or disruptions
- Look at status of dependent services (e.g. RDS, S3, DynamoDB)

→

3. ANALYZE LOGS AND METRICS
- Check CloudWatch logs
- Review metrics and alarms
- Look for errors, throttling or timeouts

→

4. VERIFY DEPENDENCIES
- Check networking, DNS, IAM and VPC components
- Validate service quotas
- Confirm connectivity between services

→

5. TAKE ACTION
- Implement failover or workaround
- Escalate if it is an AWS service issue
- Communicate with users and stakeholders

VERIQTA

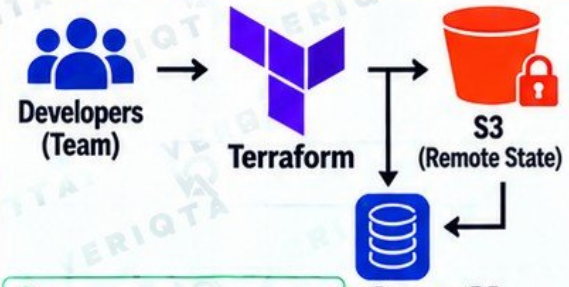
Terraform State, Modules and Infrastructure Safety

Q27. Why is Terraform state critical, and how would you protect it in a team environment?

WHY STATE IS CRITICAL

- Terraform state tracks the real infrastructure resources.
- It maps configuration to actual cloud resources.
- It is used to plan, detect changes and manage updates.
- Losing or corrupting state can cause resource duplication, deletion or drift.

EXAMPLE: REMOTE STATE IN AWS



- Store state in S3 (encrypted)
- Use DynamoDB for state locking
- Enable versioning and access control

BEST PRACTICES

- Use remote state (e.g. S3)
- Enable encryption (at rest and in transit)
- Use IAM roles with least privilege
- Enable versioning and backups
- Use state locking (e.g. DynamoDB)
- Restrict access and monitor changes

Q28. Terraform plans to replace a critical production resource unexpectedly. What would you investigate before applying?

POSSIBLE CAUSES

- Configuration change in code
- Provider version change
- Resource argument modification
- Default values changed
- State drift (manual changes)
- Infrastructure dependencies
- Module or variable changes
- Provider or API behavior change

INVESTIGATION STEPS

1. Review the terraform plan output in detail (what is changing and why).
2. Check recent code changes (e.g. variable, module, provider version).
3. Compare with the current state (terraform state show).
4. Check for manual changes in the cloud (console or CLI).
5. Look for provider or API changes in release notes.
6. Validate dependencies (resources that may force replacement).
7. Test the plan in a non production environment first.

ACTIONS BEFORE APPLY

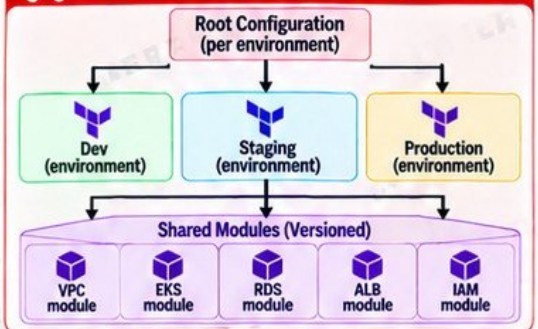
- Do not apply immediately.
- Understand the exact reason for replacement.
- Review code, variables and modules.
- Confirm there is no unintended drift or manual change.
- Check provider and dependency changes.
- Test in a staging environment.
- Get a peer review for production changes.

Q29. How would you structure Terraform modules and environments for multiple teams without creating excessive coupling?

MODULE STRUCTURE

- Use reusable and generic modules
- Keep modules focused on a single responsibility
- Use clear inputs and outputs
- Avoid hard coded values
- Use version control (e.g. semantic versioning)

ENVIRONMENT STRUCTURE



KEY PRINCIPLES

- Separate environments (dev, staging, production).
- Use remote state per environment.
- Use common, versioned modules.
- Keep modules loosely coupled.
- Use clear naming and tagging standards.
- Implement access controls per team.
- Allow teams to work independently without affecting each other.

VERIQTA

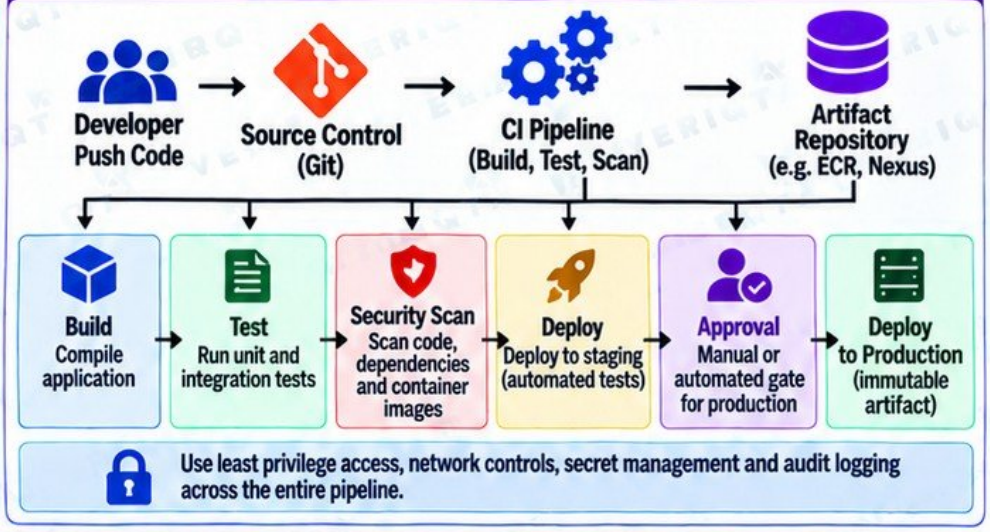
CI/CD Architecture and Pipeline Security

Q30. How would you design a secure CI/CD pipeline for a production application?

KEY DESIGN PRINCIPLES

- Use a secure, automated and repeatable pipeline.
- Implement security checks at every stage.
- Use least privilege access and short lived credentials.
- Keep environments isolated (dev, staging, production).
- Include artifact scanning and dependency checks.
- Enable approval gates for production deployments.

EXAMPLE SECURE CI/CD ARCHITECTURE



Q31. How do you prevent untrusted code from accessing production credentials in a CI/CD environment?

RISKS FROM UNTRUSTED CODE

- Malicious code can steal secrets and credentials.
- Compromised dependencies can access environment variables.
- Pull requests from forks can run untrusted code.
- Attackers can modify build scripts to exfiltrate secrets.

PREVENTION STRATEGIES

- Use short lived credentials (e.g. OIDC instead of long term keys).
- Restrict access based on branch, repo and environment.
- Use separate credentials for each environment with least privilege.
- Do not expose production secrets to build steps from untrusted sources.
- Use secret management tools (e.g. AWS Secrets Manager, HashiCorp Vault, GitHub Actions OIDC).
- Run additional security scans for dependencies and scripts.

ADDITIONAL CONTROLS

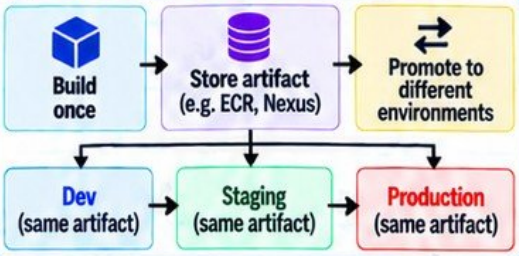
- Use environment protection rules and required approvals.
- Limit workflow permissions (sandboxed runners).
- Use self hosted runners for sensitive workloads.
- Validate and pin dependencies (checksums, versions).
- Monitor and audit pipeline activity (e.g. who accessed what).
- Use network restrictions (e.g. no direct access to production).

Q32. What is artifact promotion, and why is it safer than rebuilding an application for each environment?

WHAT IS ARTIFACT PROMOTION?

Artifact promotion means **building the application once** and promoting the **same, tested artifact** through multiple environments (dev → staging → production) without rebuilding.

HOW IT WORKS



WHY IT IS SAFER

- Ensures consistency across environments.
- Reduces the risk of environment drifts and configuration differences.
- The same tested artifact is deployed everywhere.
- Improves traceability and auditability.
- Minimizes the chance of introducing new issues during deployment.

VERIQTA

Deployment Strategies and Rollback Decisions

Q33. When would you choose blue-green deployment over canary or rolling deployment?

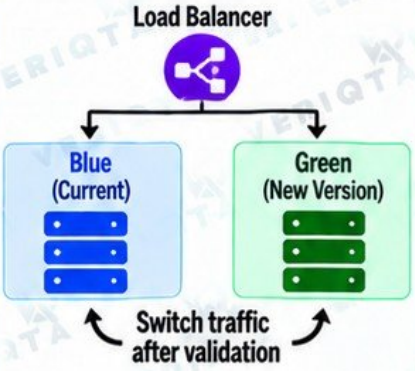
DEPLOYMENT STRATEGIES OVERVIEW

Blue-Green
Two identical environments. Switch all traffic at once.

Canary
Release to a small percentage of users first.

Rolling
Update instances gradually one at a time.

BLUE-GREEN DEPLOYMENT



WHEN TO CHOOSE BLUE-GREEN

- You need zero downtime deployments.
- You want a quick and safe rollback (if issues are found).
- The application is critical and cannot tolerate partial failures.
- You need to validate the new version in a full production-like environment.
- Infrastructure cost is acceptable (you run two environments).

Q34. A deployment passes pipeline checks but increases production error rates. How would you investigate and recover?

INVESTIGATION STEPS

1. Check monitoring and alerting (e.g. error rate, latency, saturation).
2. Compare metrics before and after deployment.
3. Review application logs and recent changes.
4. Check infrastructure dependencies (database, cache, third party services).
5. Look for configuration changes (environment variables, feature flags).
6. Validate traffic patterns and user impact.

COMMON ROOT CAUSES

- Application bug introduced in the new version.
- Configuration change causing unexpected behavior.
- Dependency or database issue (e.g. timeouts, connection limits).
- Increased traffic or load exposing a latent issue.
- Infrastructure change (e.g. instance type, scaling).

RECOVERY ACTIONS

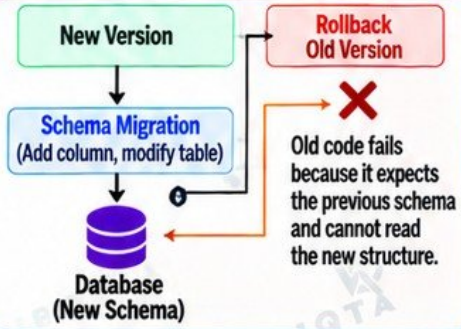
1. Roll back to the previous stable version (if safe).
2. Disable the feature using a feature flag (if available).
3. Scale up infrastructure (if it is a capacity issue).
4. Fix configuration and redeploy.
5. Mitigate at the infrastructure layer (e.g. adjust timeouts, connection limits).
6. Communicate with stakeholders and monitor until stable.

Q35. Why can rolling back application code fail to restore service after a database schema migration?

WHY ROLLBACK CAN FAIL

- Database schema migrations can make irreversible changes.
- The new version may have modified, renamed or deleted database objects.
- Old application code may not be compatible with the new schema.
- Data may have been transformed to a new format that the old code cannot read.

EXAMPLE SCENARIO



HOW TO AVOID THIS

- Use backward compatible schema changes (additive changes).
- Use feature flags and deploy code changes in a safe order.
- Separate schema migrations from application deployments.
- Test rollback in a staging environment.
- Use expand and contract pattern for schema changes.
- Have a database rollback plan (where possible).

VERIQTA

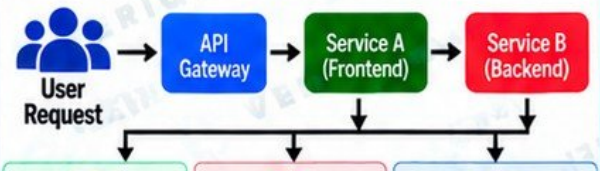
Metrics, Logs and Distributed Tracing

Q36. A microservice has high latency, but CPU and memory metrics appear healthy. How would you investigate?

INVESTIGATION STEPS

1. Check application metrics (latency, error rate, throughput).
2. Review distributed traces to find slow spans.
3. Analyze logs for timeouts, retries or downstream errors.
4. Check downstream services (database, external APIs, queues).
5. Verify network and DNS latency between services.
6. Look for throttling, rate limits or connection pool issues.

MICROSERVICE OBSERVABILITY VIEW



METRICS
- Latency
- Error rate
- Throughput
- CPU and memory
- Request count

LOGS
- Application logs
- Error messages
- Timeout events
- Retries
- Dependency errors

TRACES
- End to end request flow
- Identify slow spans
- Find failing dependency
- See time spent per service
- Check external calls

COMMON ROOT CAUSES

- Slow downstream service (e.g. database or external API)
- Increased network latency or DNS issues
- Connection pool exhaustion
- Third party rate limiting or throttling
- Inefficient queries or external API slowness
- Queue backlog or worker delay

Q37. How would you correlate metrics, logs, and distributed traces to identify a failing dependency?

CORRELATION APPROACH

1. Start with the symptom (e.g. high latency or error rate).
2. Check metrics to find which service or dependency is affected.
3. Use traces to see the full request flow and identify slow or failed spans.
4. Review logs for detailed error messages and context.
5. Correlate using timestamps, trace IDs and request IDs.
6. Confirm the failing dependency and its impact on the service.

CORRELATION FLOW

1. METRICS
High latency or error rate detected

↓

2. TRACES
Find slow span and identify downstream service

↓

3. LOGS
Check logs for errors, timeouts or failures using trace ID

↓

4. ROOT CAUSE
Confirm failing dependency

TOOLS EXAMPLE

- **Prometheus** — Metrics (latency, error rate, request count)
- **Grafana** — Visualize and correlate data
- **OpenTelemetry** — Distributed tracing (trace IDs)
- **Jaeger** — Trace visualization (service map)
- **ELK Stack (Elasticsearch, Logstash, Kibana)** — Centralized logs and search
- **CloudWatch** — Metrics, logs and traces (AWS)

Q38. What are the operational risks of high-cardinality metrics, and how would you control them?

WHAT IS HIGH CARDINALITY?

High cardinality metrics have a very large number of unique label values.
(e.g. user IDs, request IDs, IP addresses, session IDs), creating millions of unique time series.

OPERATIONAL RISKS

- Increased storage and cost
- Slower query performance
- Higher memory and CPU usage
- Can cause monitoring system instability
- Harder to troubleshoot and visualize

HOW TO CONTROL IT

- Use low cardinality labels (service, environment).
- Avoid user IDs, request IDs and unique values.
- Aggregate or sample high cardinality data.
- Use allowlists for labels.
- Set cardinality limits and monitoring alerts.
- Apply proper metric design and naming.

VERIQTA

SLOs, Error Budgets and Reliability Decisions

Q39. How do you define meaningful Service Level Indicators and Service Level Objectives for a production API?

1. CHOOSE MEANINGFUL SLIs

Select metrics that reflect real user experience.

- **Availability** — Percentage of successful requests
- **Latency** — Request duration (p50, p95, p99)
- **Error Rate** — Percentage of failed requests (4xx, 5xx)
- **Throughput** — Requests per second (RPS)
- **Freshness** — How up to date the data is (if applicable)

2. DEFINE SLOS

Set a target based on business needs, user expectations and historical data.

- **Example Availability SLO** — 99.9% of requests succeed over a 30 day period.
- **Example Latency SLO** — 95% of requests complete within 300 ms.
- **Example Error Rate SLO** — Error rate less than 0.1% over a 30 day period.
- **Example Throughput SLO** — Support at least 1,000 RPS with stable performance.

3. IMPORTANT CONSIDERATIONS

- Align SLOs with user needs and business value.
- Use realistic and achievable targets.
- Measure from the user perspective (end to end).
- Consider different endpoints and critical user journeys.
- Account for normal traffic patterns and seasonality.
- Review and adjust SLOs regularly.
- Use multiple SLIs (availability, latency and error rate) for a complete view.

Q40. Your service is consuming its error budget rapidly. How would that influence deployment and reliability decisions?

1. UNDERSTAND THE ERROR BUDGET

The error budget is the allowed amount of unreliability based on your SLO.

Example:
If your availability SLO is 99.9%, you have 0.1% error budget.



Used	Remaining
0.07%	0.03%

2. IMPACT ON DEPLOYMENT DECISIONS

If the error budget is being consumed quickly, you should:

- Pause non essential feature deployments.
- Focus on fixing reliability issues instead of adding new features.
- Tighten release approval process.
- Deploy only critical fixes with extra validation and monitoring.

3. RELIABILITY FOCUS AREAS

Use the remaining error budget to prioritize:

- Identify and fix the root cause of increased errors.
- Improve system stability and performance.
- Reduce risk by increasing testing and canary releases.
- Reduce dependencies that are causing failures.
- Increase monitoring and alerting to detect issues earlier.

4. DECISION GUIDELINES

ERROR BUDGET STATUS	DEPLOYMENT DECISION	FOCUS
Healthy (most budget remaining)	Normal deployments	Continue improving
Reducing (significant usage)	Limit to critical changes	Investigate and stabilize
Almost exhausted (very little remaining)	Pause feature deployments	Fix issues and restore reliability
Exhausted (SLO is violated)	Stop non critical deployments	Full focus on incident response and recovery

5. KEY TAKEAWAYS

- The error budget helps balance new features and reliability.
- A rapidly consuming error budget is a signal to slow down and focus on stability.
- Make data driven decisions based on real user impact.
- Communicate with stakeholders about the reliability status.
- Restore the error budget before resuming normal feature development.

VERIQTA

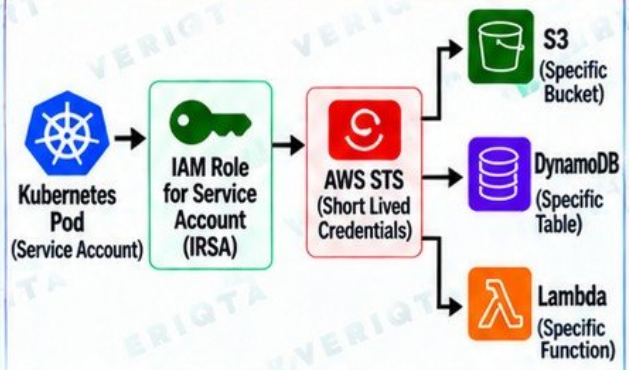
Identity, Secrets and Supply-Chain Security

Q41. How would you implement least-privilege access for Kubernetes workloads accessing cloud services?

KEY PRINCIPLES

- Use a dedicated identity for each workload (no shared keys).
- Grant only the minimum permissions required.
- Use short lived credentials instead of long lived keys.
- Scope access to specific resources and actions.
- Use namespace based isolation and policy controls.
- Monitor and audit access regularly.

ARCHITECTURE EXAMPLE (EKS + AWS)



Permissions are restricted to only the required services, resources and actions.

BEST PRACTICES

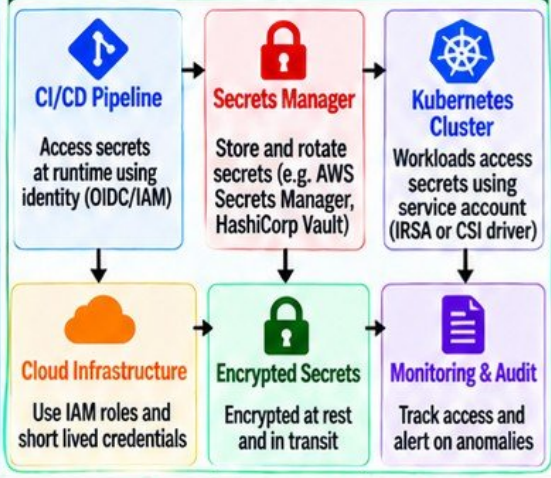
- Use IRSA or workload identity (no static keys).
- Apply least privilege IAM policies.
- Use resource level conditions (e.g. specific bucket, table or ARN).
- Rotate credentials automatically.
- Use Kubernetes network policies and admission controls.
- Enable audit logs and alerts for unusual access.

Q42. How would you secure secrets across CI/CD pipelines, Kubernetes clusters, and cloud infrastructure?

SECRET MANAGEMENT STRATEGY

1. Use a centralized secrets manager (e.g. AWS Secrets Manager).
2. Use short lived credentials and rotation.
3. Restrict access based on identity and environment.
4. Avoid storing secrets in code, images or configuration files.
5. Encrypt secrets at rest and in transit.
6. Use audit logs and alerting for secret access.

END TO END FLOW



ADDITIONAL CONTROLS

- Use environment specific secrets (dev, staging, prod).
- Apply strict access controls and approval processes.
- Use sealed secrets or external secrets operator for Kubernates.
- Implement secret rotation and automatic expiry.
- Monitor for unusual access patterns.
- Prevent secret leakage in logs, artifacts and images.

Q43. How would you reduce software supply-chain risk without making deployments unnecessarily slow?

KEY STRATEGIES

- Use trusted and verified sources for dependencies and base images.
- Implement automated vulnerability scanning (images, packages, IaC).
- Use dependency pinning and version control.
- Verify artifacts using signatures (e.g. Sigstore, cosign).
- Use policy gates to block high risk vulnerabilities only.

SUPPLY CHAIN SECURITY FLOW



BALANCING SECURITY AND SPEED

- Integrate security checks early in the pipeline.
- Use parallel scanning and caching to reduce build time.
- Focus on high and critical risks for blocking.
- Use incremental scanning for unchanged components.
- Maintain an approved list of trusted images and dependencies.
- Automate the entire process.

VERIQTA

Distributed Systems and Cascading Failures

Q44. How can retries and timeouts turn a temporary dependency slowodown into a system-wide outage?

1. WHAT HAPPENS?

1. A dependency becomes slow (higher latency).
2. Clients wait until timeout and then retry.
3. Retries increase the total number of requests.
4. The dependency becomes overwhelmed.
5. Queues build up across services (thread pools, connection pools).
6. More services start to timeout and retry, creating a cascade.
7. The entire system experiences an outage (cascading failure).

2. CASCADING FAILURE FLOW



3. HOW TO PREVENT IT

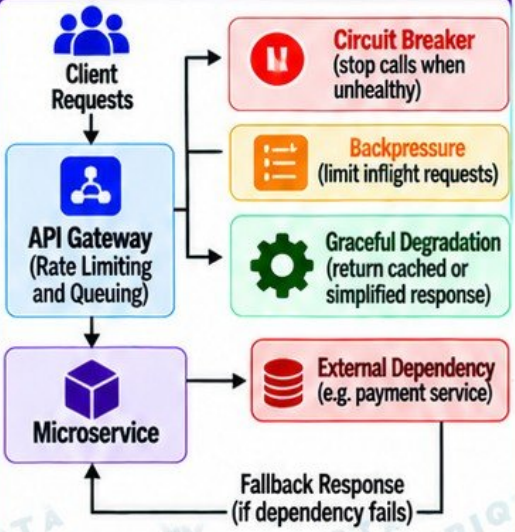
- Set sensible timeouts (short and realistic).
- Use limited and controlled retries with exponential backoff.
- Implement jitter to avoid retry storms.
- Use circuit breakers to stop calling unhealthy dependencies.
- Set connection and concurrency limits.
- Use bulkheads to isolate failures.
- Monitor latency, error rates and retry volume.
- Test failure scenarios regularly (e.g. chaos testing).

Q45. How would you design a service to handle partial failures using circuit breakers, backpressure, and graceful degradation?

1. DESIGN PRINCIPLES

- Detect and isolate failing dependencies quickly.
- Prevent the system from being overwhelmed (backpressure).
- Continue to provide essential functionality (graceful degradation).
- Maintain good user experience during partial failures.
- Recover automatically when dependencies are healthy again.

2. ARCHITECTURE EXAMPLE



3. KEY IMPLEMENTATION AREAS

- Use circuit breakers (e.g. open, half open, closed).
- Apply backpressure (queues, rate limits, concurrency limits).
- Implement graceful degradation (e.g. cached data, reduced features, default responses).
- Use timeouts and retries with exponential backoff.
- Monitor dependency health (latency, error rate, availability).
- Alert on partial failures before they impact users.
- Regularly test failure scenarios (e.g. chaos engineering).

VERIQTA

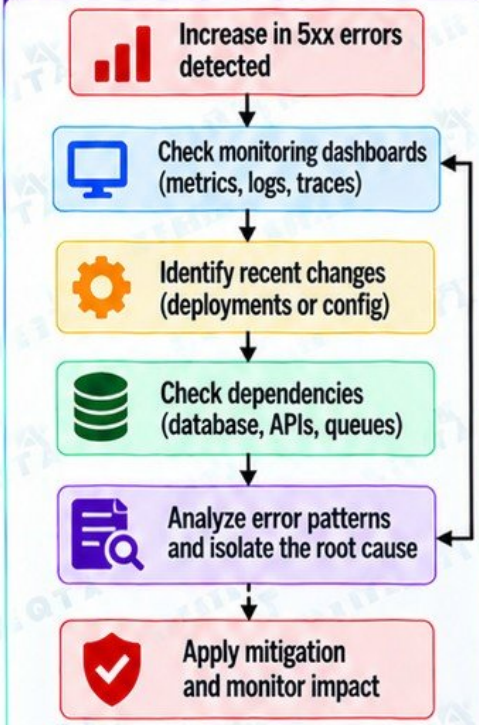
Production Incident Response

Q46. A production application experiences a sudden increase in 5xx errors. What are your first investigation and mitigation steps?

1. INITIAL INVESTIGATION STEPS

1. **Confirm the scope and impact.** Check which endpoints, regions and user segments are affected.
2. **Check recent changes.** Look for deployments, configuration changes or infrastructure updates.
3. **Review key metrics.** Check 5xx rate, latency, traffic volume and resource utilization (CPU, memory, connections).
4. **Analyze logs and traces.** Identify common error messages, failing dependencies and patterns.
5. **Check external dependencies.** Verify the health of databases, APIs, queues and third party services.
6. **Determine the root cause.** Correlate metrics, logs and traces. to isolate the issue.

2. INVESTIGATION FLOW



3. MITIGATION ACTIONS

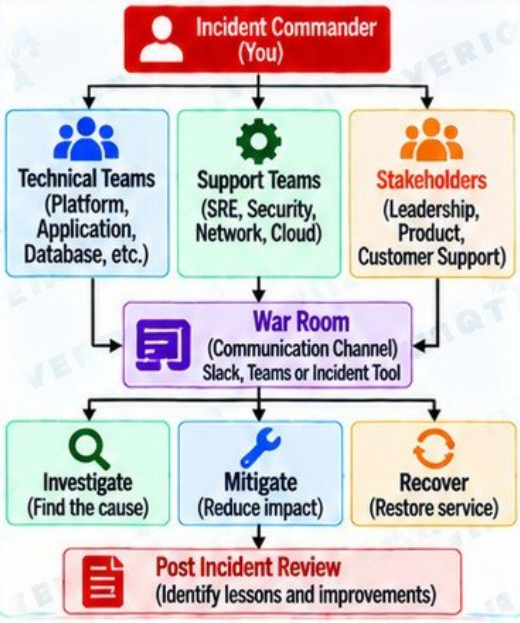
- Rollback to previous stable version if the issue is recent.
- Scale up infrastructure (if it is a capacity issue).
- Disable non critical features to reduce load.
- Apply configuration changes (e.g. increase timeouts, connection limits).
- Failover to a healthy region or availability zone (if available).
- Use circuit breakers to stop cascading failures.
- Monitor continuously to confirm recovery.
- Communicate with stakeholders about the impact and progress.

Q47. How would you lead a high-severity incident involving multiple engineering teams and unclear ownership?

1. INCIDENT LEADERSHIP STEPS

1. Take ownership and drive the incident process forward.
2. Establish a clear incident communication channel (e.g. war room).
3. Define and communicate the incident goals (stabilize, mitigate, recover).
4. Identify and engage the right teams, even if ownership is unclear.
5. Ensure regular and structured updates for stakeholders.
6. Make data driven decisions and avoid assumptions.
7. Keep calm, focused and document everything.

2. INCIDENT COMMAND STRUCTURE



3. KEY LEADERSHIP PRINCIPLES

- Establish a single source of truth for incident information.
- Focus on impact and customer experience.
- Make clear decisions, even with incomplete information.
- Delegate tasks to the right people and remove blockers.
- Maintain regular communication with all stakeholders.
- Avoid blame during the incident.
- Ensure operational, security and business impact are considered.
- Drive a post incident review to capture root cause and prevention actions.

VERIQTA

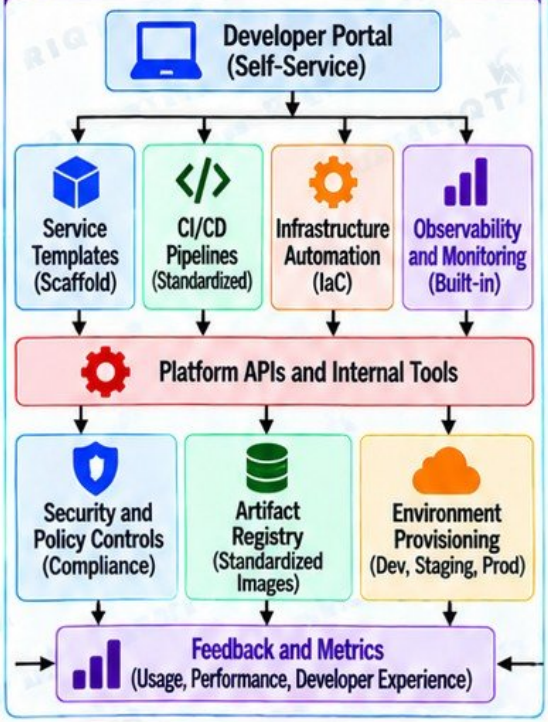
Platform Engineering and Developer Experience

Q48. How would you design an internal developer platform that improves deployment consistency without creating a bottleneck for engineering teams?

1. KEY DESIGN PRINCIPLES

1. Self-service and easy to use for developers.
2. Built in security, compliance and best practices by default.
3. Standardized and reusable components (templates).
4. Flexible enough to support different use cases.
5. Automated and scalable to avoid manual bottlenecks.
6. Clear ownership, documentation and support.
7. Continuous improvement based on developer feedback.

2. REFERENCE ARCHITECTURE



3. HOW TO AVOID BOTTLENECKS

- Automate everything (provisioning, pipelines, testing).
- Provide clear golden paths and templates.
- Allow flexibility and escape hatches for advanced use cases.
- Use well documented APIs and tooling instead of manual processes.
- Decentralize ownership with platform enablement teams.
- Continuously gather developer feedback and iterate.
- Measure adoption and friction to improve the platform.

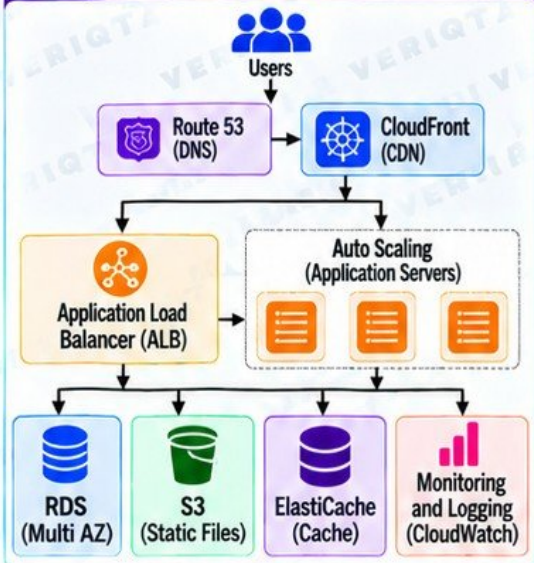
Infrastructure Architecture and Cost Trade-offs

Q49. How would you design a cost-optimized and scalable infrastructure architecture for a production application? What are the key trade-offs you would consider?

1. KEY CONSIDERATIONS

1. Business requirements (performance, availability, compliance).
2. Workload characteristics (steady, variable, regional).
3. Scalability and resilience targets.
4. Cost optimization opportunities.
5. Operational complexity and maintenance effort.
6. Security and compliance requirements.
7. Trade-offs between cost, performance and availability.

2. EXAMPLE ARCHITECTURE



3. KEY TRADE-OFFS

- Managed services vs self managed (infrastructure cost vs operational overhead).
- On demand vs reserved vs spot instances (cost vs availability).
- Single region vs multi region (cost vs resilience and latency).
- Caching (performance and cost vs data consistency).
- CDN and edge caching (cost vs freshness of content).
- Storage tiers (standard, infrequent access, archive) (cost vs access time).
- High availability vs cost (multi AZ or multi region).
- Automation and IaC (initial effort vs long term cost and consistency).

VERIQTA

Infrastructure Architecture and Cost Trade-Offs

Q49. How would you reduce cloud infrastructure costs without compromising availability, security, or performance?

1. KEY AREAS TO OPTIMIZE

1. **Compute**
   Right size, use autoscaling, use appropriate instance types.
2. **Storage**
   Use the right storage class for the use case.
3. **Networking**
   Optimize data transfer and architecture.
4. **Databases**
   Right size, optimize queries, use read replicas and serverless where appropriate.
5. **Observability**
   Control log volume and metric cardinality.
6. **Architecture**
   Use efficient, resilient and scalable architecture patterns.

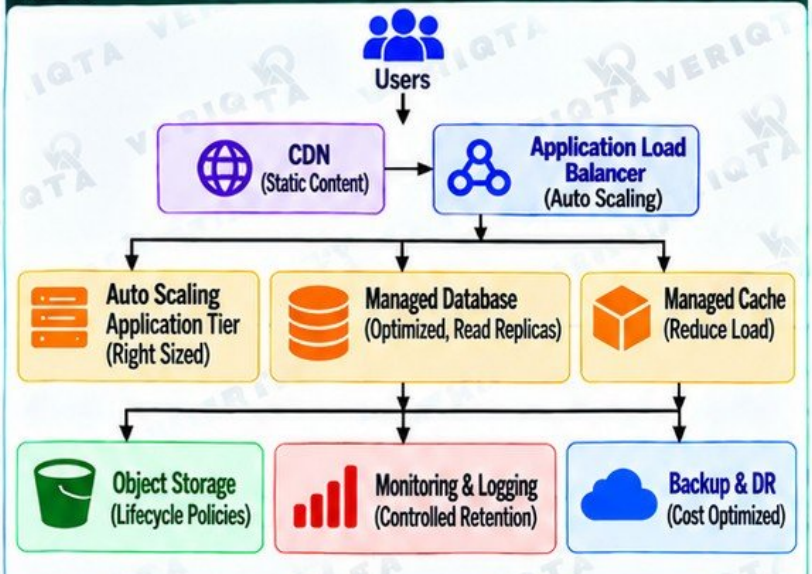
2. PRACTICAL STRATEGIES

- Use right sized instances and remove idle resources.
- Use autoscaling to match demand and avoid overprovisioning.
- Use managed and serverless services where they are cost effective.
- Use storage lifecycle policies (e.g. move to cheaper storage).
- Use content delivery networks (CDN) to reduce origin load and bandwidth costs.
- Optimize database queries and use read replicas or caching.
- Schedule non production environments (dev, test) to stop when not in use.
- Identify and remove unused resources (e.g. old snapshots, logs, load balancers, IPs).

3. MAINTAIN AVAILABILITY, SECURITY AND PERFORMANCE

- Use multi AZ and multi region only where required by business needs.
- Do not compromise security controls for cost savings.
- Monitor performance and set appropriate scaling thresholds.
- Use infrastructure as code to maintain consistency and avoid configuration drift.
- Implement proper testing before changing instance types or architectures.
- Use gradual changes and monitor impact on availability and performance.
- Regularly review costs with engineering, security and business stakeholders.

4. EXAMPLE COST OPTIMIZATION ARCHITECTURE



5. KEY TAKEAWAYS

1. Focus on right sizing and removing unused resources.
2. Use autoscaling and serverless where it makes sense.
3. Optimize storage, networking, databases and observability costs.
4. Maintain availability, security and performance while reducing costs.
5. Use data and metrics to drive decisions.
6. Implement changes gradually and monitor impact.
7. Collaborate with engineering, security and business teams.

VERIQTA

Senior DevOps Architecture and Production Readiness

Q50. You inherit a production platform with unreliable deployments, weak observability, frequent Kubernetes incidents, and rising cloud costs. How would you assess, prioritize, and improve the platform over your first 90 days?

1. INITIAL ASSESSMENT (DAYS 1–15)

1. Understand business objectives, SLAs and key stakeholders.
2. Map current architecture, services, environments and dependencies.
3. Review deployment process and recent failures.
4. Evaluate observability coverage (metrics, logs, traces and alerting).
5. Analyze Kubernetes incidents, patterns and root causes.
6. Review cloud costs, usage trends and major cost drivers.
7. Identify top risks, quick wins and gaps.

2. KEY AREAS OF FOCUS

Deployment Reliability
Stabilize and standardize delivery process.

Observability
Improve metrics, logs, traces and alerting.

Kubernetes Stability
Reduce incidents and improve cluster health.

Cost Optimization
Identify and eliminate waste without impacting performance.

Security and Compliance
Fix critical security gaps and access controls.

People and Process
Work with teams, improve documentation and ownership.

3. PRIORITIZATION STRATEGY

1. Focus on high impact and high risk areas first.
2. Deliver quick wins to build trust.
3. Align with business needs and team capacity.
4. Balance reliability, security, performance and cost.
5. Address systemic issues not just individual incidents.
6. Create a phased plan with clear milestones.

4. 90 DAY IMPROVEMENT PLAN

DAYS 1–30 STABILIZE

- Gain deep understanding of the platform.
- Fix critical issues and reduce immediate risks.
- Improve deployment reliability and rollback capability.
- Enhance observability and alerting for key services.
- Identify major cost savings opportunities.

DAYS 31–60 IMPROVE

- Implement standard deployment process (e.g. GitOps, CI CD).
- Strengthen observability across all services.
- Address recurring Kubernetes incidents (e.g. scaling, resource limits).
- Start cost optimization initiatives (rightsizing, reserved instances, etc).
- Improve documentation, runbooks and knowledge sharing.

DAYS 61–90 SCALE

- Roll out long term architectural improvements.
- Implement advanced monitoring and SLOs.
- Formalize incident response and disaster recovery.
- Continue cost optimization and measure impact.
- Build a sustainable platform with clear ownership, processes and continuous improvement.

5. SUCCESS METRICS

- Fewer failed deployments and faster recovery.
- Higher observability coverage and faster incident detection.
- Reduced Kubernetes incidents and improved cluster stability.
- Lower cloud costs while maintaining performance.
- Improved security posture and compliance.
- Higher team productivity and developer experience.

6. KEY PRINCIPLES FOR LONG TERM SUCCESS

- Automate everything possible
- Measure and track progress
- Collaborate with teams
- Focus on system design and resilience
- Balance reliability, security, performance and cost
- Continuously improve